Warum überhaupt Make?

Betrachten wir ein kleineres C Projekt, was Hello World in 3 Dateien implementiert.[1] Wir haben eine Funktion, `message`, die in `message.h` deklariert und `message.c` definiert wird. Diese gibt einen String zurück, der in `main` in `hello.c` ausgegeben wird.

```
// message.h
#ifndef MESSAGE_H
#define MESSAGE_H

const char *message(void);

#endif

// message.c
#include "message.h"

const char *
message(void)
{
  return "Hello, world!";
}

// hello.c
#include <stdio.h>

#include "message.h"

int
main(void)
{
  puts(message());
  return 0;
}
```

Das ganze kann, wie bekannt, mit `clang` kompiliert werden:

```
$ clang -c hello.c -o hello.o
$ clang -c message.c -o message.o
$ clang hello.o message.o -o hello
$ ./hello
Hello, world!
```

Allerdings `clang` bei so einem Aufruf kaum Warnungen aus, welche einige Fehler offensichtlich machen. Deshalb empfehlen wir immer mit `-Wall -Wextra` zu kompilieren.[2] Um Fehler bei der Speicherverwaltung zu vermeiden, sollte noch AddressSanitizer aktiviert werden. Also sehen diese Befehle eigentlich so aus:

```
$ clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
$ clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
$ clang -fsanitize=address hello.o message.o -o hello
```

Bei jeder Änderung müssen mindestens alle veränderten C-Dateien neu kompiliert und `hello` neu gelinkt werden—wir werden gleich sehen, warum nicht immer offensichtlich ist, welche das sind. Auch

[1] Natürlich ist das sehr komplex für Hello World, aber grundsätzlich sind 2 bis 3 Quelldateien plus Header nicht ungewöhnlich für SysProg Abgaben.

[2] Es gibt auch noch `-Weverything`, was wirklich alle Warnungen aktiviert. Es lohnt sich auch das auszuprobieren, aber meistens ist es dann sinnvoll, einzelne Warnungen wieder auszustellen, um die wirklich wichtigen nicht zu übersehen.

wenn Bash uns mit den Pfeiltasten durch alte Befehle rotieren oder mit Strg+R suchen lässt, ist das hier keine große Hilfe, da so schnell ein Schritt ausgelassen wird.

Zusätzlich sollten diese Befehle dokumentiert sein, damit niemand raten muss, wie euer Projekt kompiliert wird. Wäre es nicht schön, wenn wir diese Dokumentation ausführen lassen können, damit ein einzelner Befehl zur vollständigen Kompilation reicht?

Eine erste Makefile

Um das zu erreichen benutzen wir Make. Es gibt verschiedene Implementationen von Make, mit unterschiedlichen Erweiterungen zum POSIX Standard. Wir benutzen GNU Make. Das ist die Version, die ihr auf der vorgegebenen Version von Ubuntu und den anderen üblichen Linux-Distributionen findet, nicht aber auf macOS.[3]

Make wird von einer Makefile gesteuert, welche meistens einfach `Makefile` heißt. In dieser stehen Regeln, welche Make instruieren, wie es bestimmte Dateien erstellen kann. Regeln haben Outputs, die Dateien, die gemacht werden sollen, Inputs, die Dateien, die dafür gebraucht werden, und Befehle, die definieren, wie die Outputs aus den Inputs erstellt werden.

Eine Regel sieht wie folgt aus:

```
outputs: inputs
  command 1
  command 2
  ...
```

Mehrere Inputs oder Outputs werden mit Leerzeichen getrennt. Zu beachten ist, dass Befehle mit Tabs indentiert werden. Leerzeichen funktionieren nicht. Wenn ihr vernünftige Texteditoren benutzt, sollten diese automatisch Tabs für selbstgeschriebene Makefiles benutzen, aber kopieren aus anderen Quellen kann zu Problemen führen.

Betrachten wir nun eine Regel um `hello` zu erstellen.

Was wollen wir bauen?

```
hello
```

Was brauchen wir dafür?

```
hello.o message.o
```

Wie machen wir das?

```
clang -fsanitize=address hello.o message.o -o hello
```

Make Regel:

```
hello: hello.o message.o
  clang -fsanitize=address hello.o message.o -o hello
```

Wenn wir die Regel in unsere `Makefile` einfügen, kann Make jetzt für uns `hello` linken.

```
$ make hello
make: 'hello' is up to date.
$ rm hello
$ make hello
clang -fsanitize=address hello.o message.o -o hello
$ ./hello
Hello, world!
```

[3] Wer ein anderes nicht-Windows Betriebssystem benutzt, sollte wissen, welches Make installiert ist.

Wie wir sehen, weigert sich Make zuerst `hello` zu linken, weil es „up to date“ ist. Das heißt, dass die Inputs, also `hello.o` und `message.o`, älter als das Output, also `hello` sind. Make kümmert sich also automatisch darum, nur die Befehle auszuführen, die auch nötig sind um alle Änderungen umzusetzen.

Jetzt weiß Make zwar, wie `hello` zu linken ist, aber das reicht natürlich noch nicht um das ganze Projekt zu bauen.[4] Fügen wir also Regeln hinzu, um auch die C-Dateien zu kompilieren:

Was wollen wir bauen?
`hello.o`

Was brauchen wir dafür?
`hello.c`

Wie machen wir das?
```
clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
```

Make Regel:
```
hello.o: hello.c
  clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
```

Was wollen wir bauen?
`message.o`

Was brauchen wir dafür?
`message.c`

Wie machen wir das?
```
clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
```

Make Regel:
```
message.o: message.c
  clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
```

```
$ make hello
make: 'hello' is up to date.
$ rm hello *.o
$ make hello
clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
$ ./hello
Hello, world!
```

Wie wir sehen, merkt Make, dass die Inputs von `hello` nicht existieren und kompiliert diese zuerst.

Redundanz mit Patterns vermeiden

Die Regeln um die Objekte zu erstellen sind fast identisch. Wenn wir eine neue C-Datei in unser Projekt hinzufügen, müssen wir das ganze nochmal in die Makefile eintragen, nur mit geänderten Dateinamen. Deshalb wäre es schön, wenn wir Make sagen könnten, wie C-Dateien generell zu kompilieren sind.

Dafür sind Patterns da. Wir können Inputs und Outputs als Muster angeben. Dafür wird das Prozentzeichen benutzt. Wenn Input und Output einer Regel ein `%` enthalten, steht das für einen beliebigen Teil des Dateinamens, Stamm genannt. Mit so einer Regel können alle Dateien erstellt werden, die dem Muster folgen. In den Befehlen gibt es zwei Variablen, `$<` und `$@`, die für das Input und Output stehen.[5]

[4] Wer auch die Objekte löscht und `make hello` eingibt wird sehen, dass das nicht stimmt. Make hat ein paar eingebaute Regeln um bekannte Dateitypen zu kompilieren. Das ignorieren wir.

[5] Bei mehreren Inputs oder Outputs ist `$<` konkret das erste Input und `$@` das Output, wegen dem Make die Regel anwenden möchte. Benutzt am besten nur ein Output.

Wir können also die beiden Regeln für die Objekte durch eine einzige ersetzen:

Was wollen wir bauen?
Eine Objektdatei

Was brauchen wir dafür?
Eine C-Datei

Wie machen wir das?
```
clang -Wall -Wextra -fsanitize=address -c $< -o $@
```

Make Regel:
```
%.o: %.c
  clang -Wall -Wextra -fsanitize=address -c $< -o $@
```

```
$ rm hello *.o
$ make hello
clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
$ ./hello
Hello, world!
```

Header Dependencies

Nun wollen wir unser Programm ändern, um eine Zahl auszugeben. `message` soll diese Zahl als Parameter annehmen. Das könnten wir in `message.c` beispielsweise so umsetzen:

```
#include "message.h"

#include <stdio.h>

static char msg[10];

const char *
message(unsigned int i)
{
  snprintf(msg, sizeof(msg), "%u", i);
  return msg;
}
```

Kompilieren wir das Projekt:

```
$ make hello
clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
message.c:8:1: error: conflicting types for 'message'
    8 | message(unsigned int i)
      | ^
./message.h:4:13: note: previous declaration is here
    4 | const char *message(void);
      |             ^
1 error generated.
make: *** [Makefile:4: message.o] Error 1
```

Natürlich haben wir vergessen, die Definition in `message.h` zu ändern:

```
#ifndef MESSAGE_H
#define MESSAGE_H

const char *message(unsigned int i);
```

```
#endif
```

```
$ make hello
clang -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
```

Jetzt kompiliert das Projekt. Das heißt, es sollte auch funktionieren, richtig?

```
$ ./hello
1
```

Wir haben in `main` keine Zahl an `message` übergeben, weshalb das erste Argument von `main`, also `argc`, einfach weitergereicht wird. Aber warum hat uns `clang` nicht davor gewarnt?

Weil `hello.c` gar nicht kompiliert wurde. Schließlich hängt `hello.o` ja nur von `hello.c` ab, was sich nicht verändert hat. Deshalb sollte alles, was von C-Dateien abhängt, auch von den Headern die diese `#includen` abhängen. Der Einfachheit halber sagen wir, dass alle Objektdateien, die aus C-Dateien bestehen, von allen unserer Header abhängen:

Was wollen wir bauen?
Eine Objektdatei

Was brauchen wir dafür?
Eine C-Datei und unsere Header

Wie machen wir das?

```
clang -Wall -Wextra -fsanitize=address -c $< -o $@
```

Make Regel:

```
%.o: %.c message.h
  clang -Wall -Wextra -fsanitize=address -c $< -o $@
```

```
$ make hello
clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
hello.c:8:15: error: too few arguments to function call, single argument 'i' was not
specified
    8 |         puts(message());
      |              ~~~~~~~ ^
./message.h:4:13: note: 'message' declared here
    4 | const char *message(unsigned int i);
      |             ^       ~~~~~~~~~~~~~~
1 error generated.
make: *** [Makefile:4: hello.o] Error 1
```

Jetzt sagt uns `clang` was wir falsch gemacht haben und wir können das Problem beheben.

```
#include <stdio.h>

#include "message.h"

int
main(void)
{
  puts(message(42));
  return 0;
}
```

```
$ make hello
clang -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -fsanitize=address hello.o message.o -o hello
```

```
$ ./hello
42
```

Variablen

Nun nehmen wir an, dass in unserem Projekt ein Bug ist. Entweder haben wir gute Angewohnheiten oder sämtliches `printf`-Debuggen hat zu nichts geführt; so oder so, wir wollen jetzt einen echten Debugger benutzen. Dieser beschwert sich, dass `hello` keine Debuginformationen enthält.

`clang` gibt diese aus, wenn `-g` beim Kompilieren angegeben wird. Wir könnten das einfach zu den anderen Flags hinzufügen, aber leider funktioniert LeakSanitizer, eine Komponente von AddressSanitizer, nicht in Debuggern und das Programm startet nicht. Jetzt stehen wir vor einem Problem. Eigentlich wollen wir AddressSanitizer immer haben, wenn wir unser Programm testen, müssen es aber beim Debuggen ausstellen. Müssen wir dafür immer die Makefile verändern?

Glücklicherweise nicht, dank Variablen. Variablen können sowohl aus der Makefile als auch beim Aufruf selbst ihre Werte erhalten, wobei letztere Werte bei Konflikten überwiegen. Wir können also unsere Flags in Variablen definieren, die je nach Bedarf überschrieben werden können. Dafür brauchen wir nur zwei weitere Zeilen in der `Makefile`, die `CFLAGS` für die Compilerflags und `LDFLAGS` für die Linkerflags definieren:[6]

```
CFLAGS = -g -Wall -Wextra -fsanitize=address
LDFLAGS = -fsanitize=address
```

Nun müssen die Regeln angepasst werden, um diese Flags zu verwenden:

Was wollen wir bauen?
Eine Objektdatei

Was brauchen wir dafür?
Eine C-Datei und unsere Header

Wie machen wir das?
```
clang $(CFLAGS) -c $< -o $@
```

Make Regel:
```
%.o: %.c message.h
  clang $(CFLAGS) -c $< -o $@
```

Was wollen wir bauen?
`hello`

Was brauchen wir dafür?
`hello.o message.o`

Wie machen wir das?
```
clang $(LDFLAGS) hello.o message.o -o hello
```

Make Regel:
```
hello: hello.o message.o
  clang $(LDFLAGS) hello.o message.o -o hello
```

```
$ make hello
clang -g -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -g -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
$ make CFLAGS=-g LDFLAGS= hello
make: 'hello' is up to date.
```

[6] Wir fügen hier `-g` trotzdem hinzu, damit Tools wie `objdump` sinnvolle Funktionsnamen ausgeben.

```
$ rm hello *.o
$ make CFLAGS=-g LDFLAGS= hello
clang -g -c hello.c -o hello.o
clang -g -c message.c -o message.o
clang  hello.o message.o -o hello
```

Hier sehen wir auch schon ein mögliches Problem mit Variablen: Make weiß nicht, wenn sich Variablen ändern. So kann passieren, dass aus Versehen Dateien mit verschiedenen Flags kompiliert werden. Wir empfehlen bei Neudefinition von Variablen sämtliche Objekte zu löschen.

Apropos Objekte, wenn wir die Regel für `hello` betrachten, bemerken wir, dass die Objektdateien doppelt aufgelistet werden: Einmal als Inputs, einmal im Befehl. Beim hinzufügen einer neuen C-Datei ist es also leicht, eine der beiden nötigen Änderungen zu vergessen. Glücklicherweise können Variablen auch als Inputs oder Outputs verwendet werden. Wir definieren also eine Variable, die die Namen aller Objekte enthält und passen die Regel an, um diese zu verwenden:

```
OBJECTS = hello.o message.o
```

Was wollen wir bauen?
`hello`

Was brauchen wir dafür?
Die Objekte

Wie machen wir das?
```
clang $(LDFLAGS) $(OBJECTS) -o hello
```

Make Regel:
```
hello: $(OBJECTS)
  clang $(LDFLAGS) $(OBJECTS) -o hello
```

Wildcards und Substitution References: Portable Makefiles

Im Moment ist der Header noch manuell als Inputs für Objektdateien aufgelistet. Da ein nicht-eingetragener Header nicht selber zu Kompilierungsfehlern führt, ist es einfach zu vergessen, neue Header in die Liste einzufügen. Deshalb wäre es besser, wenn Make automatisch alle Header finden würde. Der erste Schritt dabei ist, die Namen des Headers in eine Variable, `HEADERS`, zu speichern, analog zu `OBJECTS`.

```
HEADERS = message.h
```

Was wollen wir bauen?
Eine Objektdatei

Was brauchen wir dafür?
Eine C-Datei und unsere Header

Wie machen wir das?
```
clang $(CFLAGS) -c $< -o $@
```

Make Regel:
```
%.o: %.c $(HEADERS)
  clang $(CFLAGS) -c $< -o $@
```

Das hat zunächst noch nicht geholfen (außer, dass die `Makefile` jetzt etwas übersichtlicher ist). Nun lassen wir Make aber `HEADERS` auf alle Dateien mit der Endung `.h` setzen. Dies können wir mit der `wildcard` Funktion machen. Diese nimmt ein bis mehrere Pattern, nicht mit Prozentzeichen sondern wie in Bash, und gibt alle Namen vorhandener Dateien zurück, die mindestens einem Pattern entsprechen.

Ein Funktionsaufruf sieht wie eine Variablenreferenz aus, nur dass hinter dem Namen noch die Argumente folgen. Konkret weisen wir `HEADERS` jetzt wie folgt zu:

```
HEADERS = $(wildcard *.h)
```

```
$ make
make: 'hello' is up to date.
$ touch message.h
$ make
clang -g -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -g -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
```

Dasselbe können wir auch mit den Objekten machen. Hier ist natürlich das Problem, dass nicht immer alle Objektdateien schon vorliegen, wir also nur indirekt ihre Namen erhalten können. Diese sind die Namen der dazugehörigen C-Dateien, mit der Endung von `.c` zu `.o` verändert. Wir können also die C-Dateien per `wildcard` auflisten lassen und dann eine Substitution durchführen um die Objekte zu erhalten:

```
SOURCES = $(wildcard *.c)
OBJECTS = $(SOURCES:.c=.o)
```

```
$ make clean
rm -f hello hello.o message.o
$ make
clang -g -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -g -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
```

Nun hängt unsere `Makefile` weder von den Namen der Header noch von denen der C-Dateien ab. Sie kann also auch in andere Projekte mit einer gleichen (nicht vorhandenen) Ordnerstruktur verschoben werden und Make sollte, sofern genau eine der C-Dateien eine `main` enthält und keine speziellen Compiler- oder Linkerflags nötig sind, in der Lage sein, das Projekt zu kompilieren und zu einer ausführbaren Datei namens `hello` zu linken.

PHONY Targets

Bis jetzt haben alle unsere Regeln Dateien erstellt. Auch wenn genau das der eigentliche Zweck von Make ist, können wir auch Regeln benutzen um bestimmten Befehlen oder Kombinationen von Targets Namen zu geben.

Der erste Befehl, dem wir einen Namen geben sollte, ist das Löschen der Objekte und von `hello`. Das ist u.A. sinnvoll, wenn wir andere `CFLAGS` benutzen wollen, aber auch wenn wir den Code unseres Projekts archivieren wollen. Per Konvention heißt das Output dieser Regel `clean`, was bedeutet, dass ihr in den meisten Projekten, die Make benutzen, einfach `make clean` benutzen könnt ohne wissen zu müssen, welche Dateien wo während der Kompilierung erstellt werden.[7]

Was wollen wir bauen?
Nichts, wir wollen Dateien löschen
Was brauchen wir dafür?
Nichts
Wie machen wir das?
`rm -f hello $(OBJECTS)`

[7] Wir benutzen `rm -f` damit sich `rm` nicht beschwert, wenn eine der Dateien nicht existiert.

Make Regel:

```
clean:
  rm -f hello $(OBJECTS)
```

```
$ make
make: 'hello' is up to date.
$ make clean
rm -f hello hello.o message.o
$ make
clang -g -Wall -Wextra -fsanitize=address -c hello.c -o hello.o
clang -g -Wall -Wextra -fsanitize=address -c message.c -o message.o
clang -fsanitize=address hello.o message.o -o hello
```

Die zweite Regel, die per Konvention existiert, erstellt ein Output namens `all` und hängt von allen Endprodukten—also ausführbare Dateien, Bibliotheken, etc.—des Projektes ab. Hier ist das nur `hello`, aber auch hier gilt grundsätzlich wieder, dass die meisten Makeprojekte mit `make all` vollständig kompiliert werden können. Generell ist das die erste Regel in der Makefile. Das liegt daran, dass diese ausgeführt wird, wenn kein Target angegeben wird, also Make einfach nur als `make` aufgerufen wird.

Was wollen wir bauen?
Alles

Was brauchen wir dafür?
`hello`

Wie machen wir das?
`hello` wird automatisch gelinkt, es muss also nichts weiter gemacht werden.

Make Regel:

```
all: hello
```

Es gibt noch weitere Regeln, die in vielen Makefiles aufgefunden werden. Das Make Handbuch hat eine Liste aller Targets, die GNU Projekte per Konvention haben sollten. Wir sind nicht GNU und einige dieser Targets sind unnötig für kleinere Projekte, aber beispielsweise `install` und `uninstall` können in echten Projekten sinnvoll sein—in den Abgaben natürlich eher weniger.

Make weiß natürlich nicht, dass diese Regeln keine echten Dateien erzeugen. Wenn eine Datei namens `clean` existiert, wird `make clean` entscheiden, dass `clean` jünger als seine (nicht-vorhandenen) Inputs ist und nichts mehr gemacht werden muss:

```
$ touch clean
$ make clean
make: 'clean' is up to date.
```

Auch wenn das natürlich ein sehr unwahrscheinliches Szenario ist, kann dem ganzen mit dem `.PHONY` Target entgegengewirkt werden. `.PHONY` ist einer von mehreren eingebauten Targets, die als Outputs keine echten Regeln definieren sondern Make konfigurieren. Konkret sagt `.PHONY`, dass die Regeln seiner Inputs immer ausgeführt werden sollen, auch wenn Make es sonst nicht tun würden. Fügen wir also Folgendes unserer Makefile hinzu:

```
.PHONY: all clean
```

```
$ touch clean
$ make clean
rm -f hello hello.o message.o
```

Weitere Informationen

Das hier war nur ein kleiner Überblick über ein paar der vielen Features von GNU Make. Im Handbuch sind diese beschrieben.

Wer stattdessen Makefiles schreiben möchte, die mit allen standardkonformen Implementationen von Make kompatibel sind, sollte den POSIX-Standard konsultieren.